

# UAV-X AUTONOMOUS SWARM & RESILIENT FANET

## Ground-Up System Architecture, Step-by-Step Implementation Workflow & Component Change Specification

|                                                                                    |                                                                       |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>Target Competition:</b> IIT Bombay Techfest / International UAV Swarm Challenge | <b>Document Version:</b> 2.0 (Engineering Architecture)               |
| <b>Operational Domain:</b> Post-Disaster 3D Search & Comms Relay                   | <b>Execution Model:</b> Decoupled Sim Core (30 Hz) + WebGL 3D Cockpit |
| <b>Core Objectives:</b> Zero-Collisions, Dynamic Multi-Hop Routing, High PDR       | <b>Classification:</b> Master Technical Blueprint & Roadmap           |

### 1. Executive Reset: Why Ground-Up Design is Essential

Complex multi-agent autonomous robotic systems frequently fail when features (such as 3D graphics, mesh routing, SLAM, and mission controllers) are tangled together without strict boundary contracts. When UI updates thrash the DOM, or networking depends directly on UI elements, the system becomes fragile, hard to debug, and prone to unexplained UI freezing or dropped events.

**The Ground-Up Philosophy:** We separate the entire project into **6 autonomous, independent layers**. The simulation core runs on pure Python, zero-dependency mathematical physics, and can execute 100% headless at >80 Hz. The visualization cockpit is purely a display layer connected via clean WebSocket JSON telemetry. Each layer is implemented, isolated, and unit-tested before the next layer is layered on top.

| THE 6-LAYER DECOUPLED STACK ARCHITECTURE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>[Layer 6: Visualization &amp; GCS HUD] WebGL Three.js Dual-Viewport, PFD Horizon, Chart.js Metrics, CSV Export</div><div>▲ Clean WebSocket JSON Telemetry Snapshots (30 Hz)</div><div>[Layer 5: Mission FSM &amp; Autonomous Role Allocation] 10-State MAVSDK FSM, Priority PoIs, VSM Relay Positioning</div><div>▲ Mission Commands, Role Setpoints &amp; Sensor Data Packets</div><div>[Layer 4: RF Channel, Multi-Hop Routing &amp; DTN] Friis/Log-Distance, 3D Ray-Occlusion, Dijkstra DLS, DTN Ring Buffer</div><div>▲ Dynamic Ad-Hoc Mesh Topology &amp; Active Route Paths</div><div>[Layer 3: Swarm Flocking, APF &amp; Altitude Corridors] Khatib APF, Downwash Avoidance, 4-Tier Height Stratification</div><div>▲ Kinematic Target Acceleration Vectors [ax, ay, az]</div><div>[Layer 2: 6-DOF Quadcopter Kinematics &amp; Battery] Newton-Euler 6-DOF, Velocity/Acc Clamps, Motor Thrust, Battery SoC</div><div>▲ State Integration dt: Position [x,y,z], Velocity [vx,vy,vz], Attitude [r,p,y]</div><div>[Layer 1: Environment &amp; 3D Disaster Obstacles] 700x700m Theater, Ground Datum, 3D AABB Concrete Buildings, GCS Anchor</div></div> |

### 2. Step-by-Step Implementation Workflow: What Each Layer Adds

Each subsystem below is built in strict chronological sequence. Below is the precise implementation guide, including what code to write, the underlying mathematical formulation, and the exact tangible capability it adds.

## Step 1: 3D Disaster Environment & Spatial Obstacles (`sim/environment.py`, `sim/obstacles.py`)

**What is implemented:** A bounded 3D workspace (e.g., 700m × 700m × 130m) with fixed GCS origin coordinate [0, -250, 0]m. Obstacles are modeled as Axis-Aligned Bounding Boxes (AABB) defined by minimum vector  $p_{min} = [x1, y1, z1]$  and maximum vector  $p_{max} = [x2, y2, z2]$ . Each building has a specific physical material (reinforced concrete, metal composite) and an RF attenuation index (e.g. +22 dB).

**What change it adds to the system:**

- Establishes the spatial boundary ground truth.
- Introduces physical barriers that prevent drones from flying through structures.
- Provides the geometric occluders necessary for RF shadowing and line-of-sight calculation in Layer 4.

## Step 2: 6-DOF Quadcopter Kinematics & Battery Dynamics (`sim/drone.py`)

**What is implemented:** Continuous Newton-Euler rigid body equations of motion. Drones cannot teleport; they must accelerate, turn, and bank realistically based on physical constraints:

$v(t + dt) = \text{clip}(v(t) + a(t) * dt, -v_{max}, v_{max})$  with  $v_{max} = 15.0$  m/s.

$p(t + dt) = p(t) + v(t + dt) * dt$  with acceleration clamped to  $a_{max} = 5.0$  m/s<sup>2</sup>.

Energy expenditure is modeled dynamically:  $P_{total} = P_{hover} + P_{transit} * ||v||^3 + P_{avionics} + P_{rf}$ . When battery State-of-Charge (SoC) drops below 20%, the drone automatically transitions to Return-To-Launch (RTL).

**What change it adds to the system:**

- Replaces artificial instant waypoint jumping with smooth, physically authentic trajectories.
- Enforces battery endurance limits, creating real operational urgency for mission completion.
- Generates pitch/roll/yaw attitude angles used to drive the 3D visual cockpit and Primary Flight Display (PFD).

## Step 3: Swarm Separation, Flocking & 4-Tier Altitude Stratification (`sim/planning.py`)

**What is implemented:** A hybrid collision avoidance system combining **altitude corridors** with **Khatib Artificial Potential Fields (APF)**:

1. *Altitude Stratification:* Flight levels are partitioned into dedicated corridors: Launch [0–20m], PoI Survey [25–45m], Transit [50–65m], and Relay Backbone [70–90m].
2. *Khatib APF:* Attractive gradient towards target  $F_{att} = -k_{att} * (p - p_{target})$ ; repulsive gradient from obstacles and peer drones  $F_{rep} = k_{rep} * (1/d - 1/d_0) * (1/d^2) * \text{grad}(d)$  for  $d < d_0$ .
3. *Aerodynamic Downwash Repulsion:* An asymmetric vertical repulsion cylinder prevents trailing UAVs from entering the turbulent wake below a leading UAV.

**What change it adds to the system:**

- Eliminates mid-air drone collisions even in dense swarm formations (8 to 16 UAVs).
- Separates relay drones from survey drones vertically, maximizing communications coverage while keeping survey corridors clear.

## Step 4: Wireless Channel, 3D Ray-Occlusion & Dynamic Multi-Hop Routing (`sim/channel.py`, `sim/network.py`)

**What is implemented:** A dual-band RF propagation engine (2.4 GHz high-bandwidth WiFi + 915 MHz long-range LoRa) combined with 3D ray-AABB slab intersection:

1. *Free Space Path Loss (Friis):*  $PL_0 = 20 * \log_{10}(4 * \pi * f / c) = 40.05$  dB at 1 meter.
2. *Log-Distance Shadowing:*  $PL(d) = PL_0 + 10 * \eta * \log_{10}(d) + X_{sigma}$  (where  $\eta_{LoS} = 2.05$ ,  $\eta_{NLoS} = 3.60$ ).
3. *Ray-AABB Occlusion:* If a direct line-of-sight ray intersects a concrete building, a +22 dB penetration penalty is applied, dropping SNR below threshold and severing direct links.
4. *FANET-DLS Dijkstra Routing:* Edge cost is computed as  $\text{Cost} = \text{Distance} * (1 + (\text{SNR}_{thresh} - \text{SNR})) + \text{ObstaclePenalty}$ . Dijkstra shortest-path algorithm dynamically discovers multi-hop routes (e.g., UAV\_6 → UAV\_5 → RELAY\_3 → GCS).
5. *DTN Store-and-Forward:* Drones carry a 250-packet FIFO ring buffer to cache survey data during link dropouts, automatically flushing when reconnecting.

**What change it adds to the system:**

- Drones behind skyscrapers can transmit live survivor data to GCS even when direct connection is impossible.

- Demonstrates genuine self-healing network resilience: if an intermediate relay drone fails, the topology dynamically recalculates within 1–2 seconds.
- DTN ensures zero data loss during temporary communications blackouts.

### Step 5: Mission Manager, Pol Task Allocation & Virtual Spring Mesh (VSM) (`sim/mission.py`)

**What is implemented:** Autonomous swarm coordination and task distribution:

1. *MAVSDK-Compliant FSM*: Each drone executes an autonomous state machine (IDLE → TAKEOFF → TRANSIT → SURVEYING → RTL → LANDED).
2. *Pol Generation & Priority Queue*: Disaster sites (Survivor Search, Structural Collapse, Gas Hazard) prioritized by urgency.
3. *Dynamic Fleet Allocation*: Surveyors fly to Pols, perform high-resolution dwell sweeps (10s), and generate telemetry packets.
4. *Virtual Spring Mesh (VSM) Relays*: Relay UAVs act as virtual nodes connected by simulated spring-damper forces to GCS and active Surveyors. They automatically float into optimal midpoint bridging positions to maintain wireless connectivity.

**What change it adds to the system:**

- Eliminates manual pilot control; the swarm executes the entire disaster reconnaissance mission autonomously.
- Relay drones automatically position themselves where they are needed most without human intervention.

### Step 6: High-Performance WebGL 3D Cockpit & Telemetry HUD (`vis/server.py`, `vis/static/`)

**What is implemented:** A dual-viewport WebGL Ground Control Station served over FastAPI and WebSocket:

1. *Viewport 1 (External Reality)*: 3D quadcopters with spinning propellers, building models, downwash particles, and glowing multi-hop RF communication tubes.
2. *Viewport 2 (SLAM Perception)*: Real-time 3D LiDAR point cloud sweep, OctoMap 3D occupancy voxel grid, and Khatib APF force vectors.
3. *Glassmorphic HUD*: Primary Flight Display (PFD) artificial horizon, live telemetry table, active routing matrix, and Chart.js scientific analytics drawer.
4. *Non-Blocking UI Architecture*: In-place DOM updates prevent card recreation and click dropouts; dedicated toggle buttons and ESC key shortcuts ensure responsive controls.

**What change it adds to the system:**

- Transforms raw Python numerical simulations into an interactive, world-class command cockpit suitable for competition judges and investors.
- Allows real-time inspection of any drone's airspeed, battery, attitude, and routing with zero UI lag.

### 3. Subsystem Change & Impact Specification Matrix

The following matrix summarizes each component, the core algorithm implemented, the files affected, and the exact operational capability it adds to the overall autonomous swarm.

| Subsystem                  | Core Algorithm / Math                                                                 | Source Modules                         | Exact Capability Added                                                                                        |
|----------------------------|---------------------------------------------------------------------------------------|----------------------------------------|---------------------------------------------------------------------------------------------------------------|
| 1. Environment & Obstacles | 3D AABB Slab intersection, Coordinate bounds [-350, 350]m                             | sim/environment.py<br>sim/obstacles.py | Defines realistic post-disaster operational zone with multi-story buildings that block line-of-sight.         |
| 2. 6-DOF Kinematics        | Newton-Euler dynamics, Euler attitude, velocity/accel clamping                        | sim/drone.py<br>sim/types.py           | Prevents teleportation; drones fly with authentic inertia, bank angles, and speed limits (15 m/s).            |
| 3. Battery & Energy        | Nonlinear power draw: $P_{base} + P_{transit} \cdot v^3 + P_{rf}$ , SoC depletion     | sim/drone.py                           | Enforces mission endurance; triggers autonomous RTL when battery drops below 20%.                             |
| 4. Flocking & Avoidance    | Khatib APF gradient, Reynolds separation, Downwash repulsion                          | sim/planning.py<br>sim/drone.py        | Guarantees zero mid-air collisions; safe minimum clearance $\geq 1.5$ m maintained across 16 drones.          |
| 5. Altitude Corridors      | 4-tier height slicing: Launch [0-20m], Pol [25-45m], Transit [50-65m], Relay [70-90m] | sim/drone.py<br>sim/types.py           | Deconflicts airspace; survey drones stay below high-altitude relay mesh backbone.                             |
| 6. RF Propagation          | Friis FSPL ( $PL_0 = 40.05$ dB) + Log-distance path loss ( $\eta_{NLoS} = 3.60$ )     | sim/channel.py                         | Computes realistic signal-to-noise ratio (SNR) based on distance and transmission frequency.                  |
| 7. 3D Ray-Occlusion        | Slab ray-tracing through building AABBs, +22 dB attenuation penalty                   | sim/channel.py<br>sim/obstacles.py     | Forces multi-hop: drones behind tall buildings lose direct line-of-sight to GCS.                              |
| 8. Multi-Hop Routing       | Dynamic Link-State Dijkstra (FANET-DLS) with composite SNR-distance cost              | sim/routing.py<br>sim/network.py       | Self-healing mesh: finds optimal multi-hop paths (e.g. UAV_6 $\rightarrow$ UAV_5 $\rightarrow$ GCS) in < 2ms. |
| 9. DTN Buffering           | 250-packet FIFO ring buffer, store-and-forward queueing                               | sim/packets.py<br>sim/network.py       | Zero packet loss: sensor data is cached during temporary link breaks and flushed upon reconnect.              |
| 10. VSM Relay Mesh         | Virtual Spring Mesh equilibrium, spring-damper midpoint positioning                   | sim/planning.py<br>sim/mission.py      | Relay drones automatically position themselves halfway between GCS and frontline survey drones.               |
| 11. 3D LiDAR & SLAM        | Multi-beam ray-cast LiDAR sweeps, OctoMap 3D occupancy voxel grid                     | sim/mapping.py                         | Autonomous perception: drones build real-time 3D voxel maps of collapsed rubble in flight.                    |
| 12. EKF Estimation         | 9-state Extended Kalman Filter (pos, vel, att) with sensor noise injection            | sim/sensors.py                         | Simulates realistic noisy IMU/GPS fusion; tracks estimation convergence (< 0.15m error).                      |
| 13. 3D WebGL Cockpit       | Three.js hardware-accelerated dual-viewport, glowing RF tubes, particle systems       | vis/server.py<br>vis/static/           | Interactive visual command center: displays reality vs SLAM perception side-by-side at 30 Hz.                 |
| 14. Primary Flight Display | HTML5 Canvas artificial horizon, pitch/roll ladder, airspeed tape, altimeter          | vis/static/js/cockpit.js               | Provides cockpit-style avionics instrumentation for any selected drone.                                       |
| 15. Scientific Analytics   | Chart.js live plots: EKF error, PDR %, RF SNR vs distance, battery curves             | vis/static/js/cockpit.js               | Provides scientific and defense-grade performance proofs during competition judging.                          |

### 4. Step-by-Step Execution & Quality Assurance Guide

To ensure the system works with 100% stability and zero regressions, follow this testing and launch sequence:

#### Step 4.1: Run the 4-Tier Automated E2E Test Suite

Before launching the visualizer, always run the headless test suite. It executes 138 opaque-box assertions across physical dynamics, RF formulas, and multi-hop routing in less than 3 seconds:

```
python3 tests/e2e/test_runner.py
```

- Tier 1: Feature Isolation (60 tests) → 100% PASS
- Tier 2: Boundary Value Analysis (60 tests) → 100% PASS
- Tier 3: Pairwise Combinations (12 tests) → 100% PASS
- Tier 4: Real-World Disaster Scenarios (6 tests) → 100% PASS

### Step 4.2: Run the Unified Full Test Suite (257 Tests)

Runs all unit and subsystem integration tests (EKF, weather, dual-band MAC, mapping):

```
python3 -m pytest tests/ -v
```

### Step 4.3: Launch the Interactive 3D WebGL Cockpit

Launches the FastAPI WebSocket telemetry streaming server and opens the 3D cockpit:

```
python3 run_simulation.py --port 8000
```

Open your browser to: **http://localhost:8000**

---

**Summary & Next Actions:** *This blueprint defines the exact mathematical, architectural, and component-level specifications for the UAV-X Autonomous Swarm. All code modules are decoupled, verified by 257 automated tests, and ready for demonstration and competition review.*